

Deploying the Express.js Backend on AWS

Options, costs and best practices for `backend/expressJs/expressJs_ecommerce_backend`

Recommendation: run it on **one small Lightsail server (about \$5-7/month)**. It needs no code changes, gives you a fixed monthly price, and is easy to manage. If you want it nearly free, the best option is **AWS Lambda**, but it needs small code changes first.

Your backend is a normal long-running Express server: it calls `app.listen`, connects to MongoDB and creates the admin when it starts. Images go to Cloudinary, and the database is MongoDB Atlas, which stays where it is. That means you only need AWS to run the Node.js server and give it a public **HTTPS** address. HTTPS is required because Android release apps and Stripe webhooks will not use plain `http`.

1. Ways to run it on AWS

Prices are rough estimates for low traffic in the Mumbai region (ap-south-1). Always check the AWS Pricing Calculator.

#	Option	What it is	Code changes	HTTPS	Approx. cost
1	Lightsail server	A small Linux server at a fixed monthly price	None	Free certificate via Caddy, needs a domain	\$5-7/mo
2	Lightsail container service	Lightsail runs your Docker image	Add a Dockerfile	Built in, no domain needed	~\$7/mo (Nano)
3	EC2	A plain virtual server	None	Caddy, or a load balancer (+~\$16/mo)	~\$8-12/mo (t4g.micro + public IP + disk)
4	Elastic Beanstalk	AWS sets up EC2 for you	Minor	Load balancer, or set it up yourself	~\$10/mo single server, ~\$25+ with a load balancer
5	App Runner	Give it code or a container, AWS runs it	Dockerfile, or set build/start commands	Built in	~\$5-15/mo
6	Lambda (Function URL or API Gateway)	Your code runs only when a request arrives	Yes: export a handler, reuse the DB connection	Built in	~\$0-2/mo at low traffic
7	ECS Fargate + load balancer	Industry-standard containers	Dockerfile	Built in, via the load balancer	~\$35-50/mo
8	EKS (Kubernetes)	Large-scale container hosting	Lots	Built in	\$70+/mo - too much for this project

About the free tier: AWS changed it in July 2025. New accounts now get **up to \$200 in credits for about 6 months**, instead of the old "12 months free". Lightsail has also offered free first months on some plans. Check what your account gets.

2. Which to choose

Cheapest and simplest: Option 1 - Lightsail server

Fixed price, nothing to rewrite, and you learn real server setup (Nginx/Caddy, PM2, SSH), which looks good on a resume. Downsides: you install operating-system updates yourself, and there is no automatic scaling.

Near-free: Option 6 - Lambda

You pay almost nothing when nobody is using it. Downsides:

- **Slow first request** ("cold start"): the first request after an idle period takes about 1-2 seconds.
- **Atlas access**: Lambda has no fixed IP, so Atlas has to allow connections from any IP (0.0.0.0/0).
- **Code changes**: `app.listen` and the start-up code need small changes.

Industry best practice: Option 7 - ECS Fargate

- The app runs in a Docker container, with images stored in ECR.
- Secrets live in AWS Secrets Manager.
- Logs go to CloudWatch.
- GitHub Actions deploys each update automatically.

Right for a real business, but about \$40/month is too much for a portfolio project.

Middle ground: Options 2 and 5

They follow the container approach without the load balancer cost. Lightsail containers at about \$7/month also give you an HTTPS address without buying a domain.

3. Steps for the recommended setup (Lightsail server)

- 1 **Create the server.** In Lightsail, choose the Mumbai region and Ubuntu 24.04, use the **1 GB plan** (512 MB is tight for building TypeScript), and attach a **Static IP**.

- 2 **Install Node.js 22, git and PM2** (PM2 keeps the server running and restarts it if it crashes):

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs git
sudo npm i -g pm2
```

- 3 **Get the code and install packages:**

```
git clone https://github.com/Ritiksharmaji/Ecommerce-ReactNative-Application.git
cd Ecommerce-ReactNative-Application/backend/expressJs/expressJs_ecommerce_backend
npm ci
```

- 4 **Create the .env on the server**, copying `.env.example` and filling in the values. Never commit it.

- 5 **Build and start.** Your `tsconfig.json` already outputs to `dist/`, so run compiled JavaScript in production instead of `tsx`:

```
npm run build
pm2 start dist/server.js --name ecommerce-api
pm2 save && pm2 startup
```

- 6 **Add HTTPS with Caddy.** Point a domain or subdomain at the static IP. If you don't want to buy one, a free DuckDNS name works. Install Caddy and give it this one config line - Caddy gets and renews the certificate for you:

```
api.yourdomain.com {
    reverse_proxy localhost:3000
}
```

- 7 **Firewall:** open ports 80 and 443 only - never 3000 or 27017.

- 8 **MongoDB Atlas:** under Network Access, allow only the server's static IP.

- 9 **Point everything at the new URL:**

- `VITE_BACKEND_URL` for the web app.

- `EXPO_PUBLIC_API_URL` for the mobile app; rebuild the APK afterwards.
- The Stripe webhook: `https://api.yourdomain.com/api/stripe`.
- The Postman `baseUrl`.

10 To update later: run this on the server (a GitHub Action can do it automatically on every push to main):

```
git pull && npm ci && npm run build && pm2 restart ecommerce-api
```

4. Do this first, whichever option you pick

- **Rotate the secrets** that were committed to GitHub: the MongoDB password, Cloudinary secret, Stripe key and `JWT_SECRET`. Use a long random `JWT_SECRET` and a strong admin password, not `admin@123`.
- **Give this backend its own Atlas database**, so the old Render backend can't write old-format data into it.
- **Remove the [login] debug lines** in `authController.ts`. The masked request logging can stay.
- **Start with** `npm run build` and `node dist/server.js`, not `tsx`, in production. Run `npm run build` once locally first to check it compiles. Your backend already reads `PORT` from the environment.
- **Store secrets safely**. On a server, a locked-down `.env` file is fine. On ECS or Lambda, use SSM Parameter Store (free) or Secrets Manager (about \$0.40 per secret per month).
- **Host the web app separately**, on S3 + CloudFront or Amplify Hosting, which costs a few cents a month at this size. It is static files, not a server.

Prices and free-tier details change over time; confirm them in the AWS Pricing Calculator before you deploy.